

UNINASSAU

Fábrica de Software e Tópicos Avançados em Ciência da Computação
Turma 8NA | Semestre 2026.2

RotaCerta

Documento Técnico Cumulativo das Sprints 01 e 02

Grupo [INFORMAR]

Álvaro Jordão | 01748200

Arthur Sales | 01593811

Vinícius Trigueiro | 01794959

2026

Apresentação

Este documento reúne as entregas da Sprint 01 e da Sprint 02 do projeto RotaCerta. A primeira parte registra o problema, os objetivos, os requisitos e o planejamento inicial. A segunda parte apresenta a arquitetura, os modelos de software e dados, os protótipos, a estrutura inicial do banco de dados e a organização do repositório que servirão de base para a implementação.

A revisão incorpora as orientações recebidas após a Sprint 01. O cronograma passa a adotar cadência semanal, com exceção do período estendido da Sprint 02 informado pela professora. A comparação entre as versões sequencial e paralela do motor de otimização passa a ter métricas, protocolo de medição e uma primeira implementação executável com testes automatizados.

Sumário

Parte I Sprint 01 Planejamento do projeto

- 1 Identificação da equipe
- 2 Tema, problema, objetivos e público alvo
- 3 Requisitos funcionais e não funcionais
- 4 Casos de uso e Product Backlog
- 5 Cronograma semanal revisado

Parte II Sprint 02 Arquitetura e modelagem

- 6 Arquitetura do sistema
- 7 Diagrama de classes
- 8 Modelo Entidade Relacionamento
- 9 Modelo relacional
- 10 Protótipos das telas
- 11 Banco de dados e projeto no GitHub
- 12 Avaliação sequencial e paralela
- 13 Aderência às orientações da disciplina

Parte I Sprint 01 Planejamento do projeto

1 Identificação da equipe

Integrante	Matrícula	Responsabilidade inicial
Álvaro Jordão	01748200	Scrum Master e Product Owner; organização, requisitos e priorização
Arthur Sales	01593811	Backend e núcleo de otimização e paralelização
Vinícius Trigueiro	01794959	Frontend, banco de dados e documentação

Projeto RotaCerta. Turma 8NA. As responsabilidades organizam o trabalho, mas todos os integrantes participam das decisões, implementação, testes e documentação.

2 Tema, problema, objetivos e público alvo

Tema. Logística e otimização de processos para pequenos negócios com operação própria de entrega.

Problema. Farmácias, mercados, restaurantes e pequenas distribuidoras frequentemente planejam entregas de forma manual. Essa prática aumenta a distância percorrida, o consumo de combustível e a possibilidade de atraso quando o volume de pedidos cresce.

Objetivo geral. Desenvolver um sistema web que distribua pedidos entre entregadores e determine uma sequência de paradas que reduza a distância e o tempo total das rotas.

Resultados esperados. Centralizar pedidos e entregadores, reduzir distância e tempo em relação à alocação manual, calcular rotas para vários entregadores e disponibilizar uma interface responsiva para a operação diária.

Público alvo. Administradores, atendentes e entregadores de pequenos negócios locais. Os clientes finais são beneficiados por entregas mais previsíveis.

3 Requisitos funcionais e não funcionais

Requisitos funcionais

ID	Descrição
RF01	Cadastrar e autenticar usuários com perfis de administrador, atendente e entregador.
RF02	Cadastrar pedidos com cliente, endereço, prioridade e horário desejado.
RF03	Cadastrar entregadores com disponibilidade e capacidade de carga.
RF04	Gerar automaticamente uma rota otimizada para cada entregador.
RF05	Atribuir pedidos a entregadores de forma manual ou automática.
RF06	Exibir a rota como lista ordenada de paradas e mapa.
RF07	Atualizar o status do pedido entre pendente, atribuído, em rota e entregue.
RF08	Manter o histórico de entregas realizadas.
RF09	Gerar relatório com tempo, distância e comparação da otimização.
RF10	Disponibilizar painel administrativo da operação do dia.

Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Desempenho	Executar o cálculo em tempo aceitável para vários entregadores e permitir processamento paralelo.
RNF02	Segurança	Autenticar usuários, autorizar por perfil e armazenar senhas com hash seguro.
RNF03	Usabilidade	Manter a interface simples para usuários com diferentes níveis de letramento técnico.
RNF04	Portabilidade	Funcionar em navegadores e adaptar o layout para celulares.
RNF05	Escalabilidade	Suportar o crescimento de pedidos e entregadores sem degradação significativa.
RNF06	Confiabilidade	Preservar pedidos, rotas e histórico com integridade referencial.
RNF07	Manutenibilidade	Manter código organizado, documentado, testável e versionado.

4 Casos de uso e Product Backlog

ID	Ator	Caso de uso
UC01	Atendente ou administrador	Cadastrar pedido
UC02	Administrador	Cadastrar entregador
UC03	Administrador	Gerar rotas para os pedidos do dia
UC04	Entregador	Visualizar a rota atribuída
UC05	Entregador	Atualizar o status da entrega
UC06	Administrador	Consultar métricas das rotas
UC07	Todos os perfis	Autenticar-se no sistema

Product Backlog inicial

Alta prioridade. Autenticação e perfis, CRUD de pedidos, CRUD de entregadores, banco de dados e motor sequencial com vizinho mais próximo e 2-opt.

Média prioridade. Visualização das rotas, atualização de status e paralelização para múltiplos entregadores.

Evolução. Relatórios, painel consolidado, notificações e avaliação futura de C++ com OpenMP ou CUDA.

5 Cronograma semanal revisado

O cronograma abaixo substitui a periodicidade quinzenal apresentada inicialmente. A Sprint 02 mantém o prazo excepcional de 19 de setembro comunicado pela professora; as etapas seguintes usam ciclos semanais.

Sprint	Período	Entrega principal
1	03/09 a 05/09	Planejamento, requisitos e backlog
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura do projeto
3	20/09 a 26/09	Ambiente executável e contrato inicial da API
4	27/09 a 03/10	Autenticação e autorização por perfil
5	04/10 a 10/10	CRUD de clientes e pedidos
6	11/10 a 17/10	CRUD de entregadores e disponibilidade
7	18/10 a 24/10	Atribuição de pedidos e visualização de rotas
8	25/10 a 31/10	Vizinho mais próximo em modo sequencial
9	01/11 a 07/11	2-opt e linha de base de desempenho
10	08/11 a 14/11	Paralelização para múltiplos entregadores
11	15/11 a 21/11	Experimentos sequencial versus paralelo
12	22/11 a 28/11	Relatórios, testes e documentação
Final	29/11 a 05/12	Correções, vídeos e preparação para a banca

Parte II Sprint 02 Arquitetura e modelagem

A Sprint 02 transforma os requisitos em uma base técnica verificável. A solução adota uma arquitetura web modular, um modelo de dados relacional e um componente de otimização isolado, permitindo implementar e medir versões sequencial e paralela sem duplicar regras de negócio.

6 Arquitetura do sistema

Camada	Tecnologia	Responsabilidade
Interface	React, TypeScript e Vite	Aplicação responsiva para administração e operação móvel
API	FastAPI e Python 3.12	Endpoints REST, validação, OpenAPI e integração com o otimizador
Persistência	PostgreSQL 16	Integridade, índices, histórico e métricas de execução
Mapa	Leaflet, OpenStreetMap e adaptador OSRM	Exibição de rotas e matriz de distâncias sem prender o domínio a um provedor
Otimização	Python, 2-opt e processos independentes	Linha de base sequencial e cálculo paralelo por entregador
Ambiente	Docker Compose	Inicialização reproduzível de frontend, API e banco

Arquitetura do RotaCerta

Comunicação síncrona por API REST e processamento isolado de otimização

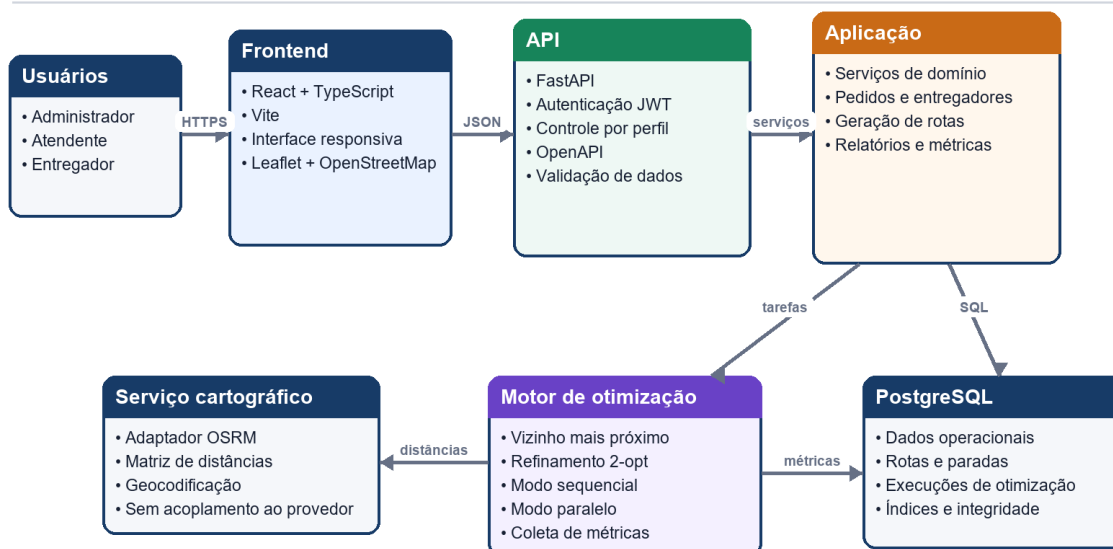


Figura 1 Arquitetura lógica e comunicação entre os componentes

O navegador envia requisições HTTPS em JSON para a API. A API executa regras de negócio, persiste dados no PostgreSQL e aciona o motor de otimização. O motor recebe pedidos, entregadores e uma matriz de distâncias por meio de uma interface estável. A mesma interface atende aos modos sequencial e paralelo, o que permite comparar desempenho sem alterar os dados de entrada.

A escolha de Python não pressupõe o uso de inteligência artificial. As orientações da disciplina aceitam inteligência artificial e/ou otimização de processamento, paralelização e alto desempenho. No RotaCerta, o componente avançado é a otimização de rotas integrada ao fluxo principal. React e TypeScript permanecem restritos à interface, enquanto Python executa o núcleo computacional.

7 Diagrama de classes

Diagrama de classes

Classes de domínio e serviço previstas para as próximas Sprints

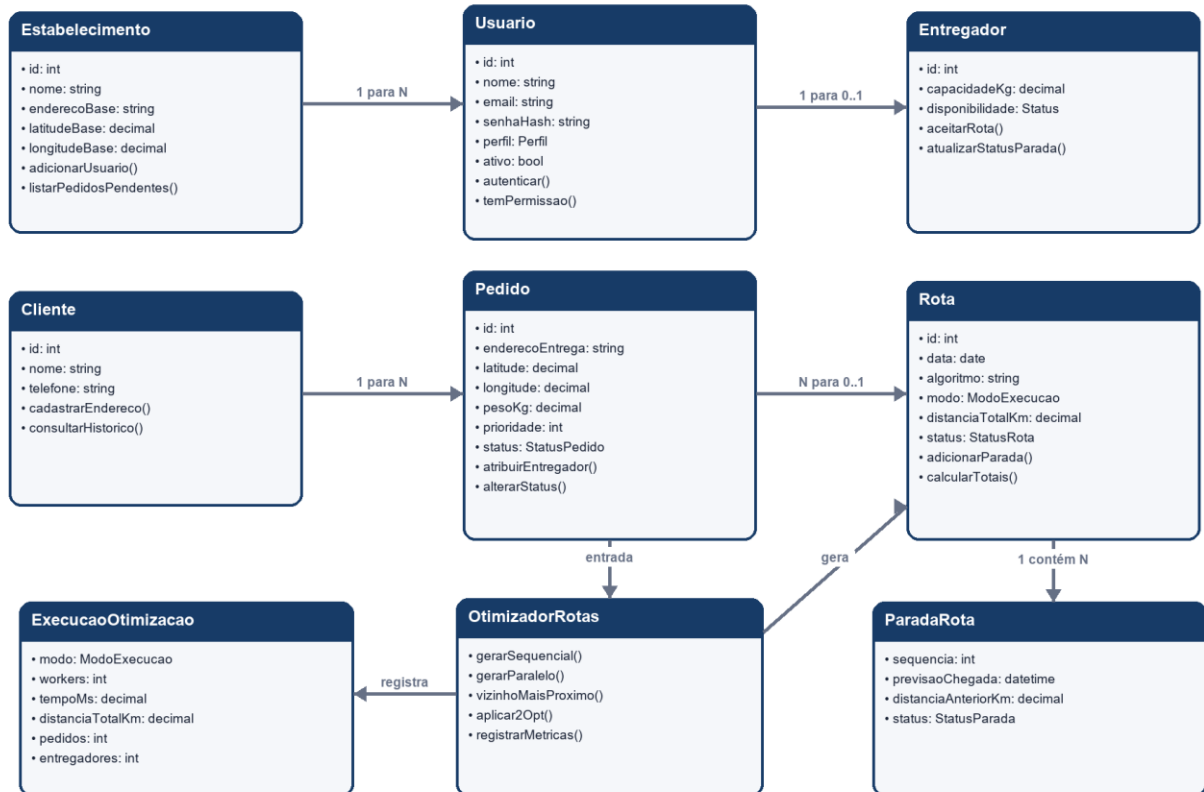


Figura 2 Classes de domínio e serviço previstas

Estabelecimento delimita os dados de cada negócio. Usuario representa autenticação e perfil; Entregador complementa um usuário operacional com capacidade e disponibilidade. Pedido registra destino, prioridade, janela de horário e status. Rota agrega paradas ordenadas. OtimizadorRotas executa o mesmo algoritmo nos modos sequencial e paralelo e registra cada ExecucaoOtimizacao.

8 Modelo Entidade Relacionamento

Modelo Entidade Relacionamento

Entidades, chaves e cardinalidades da estrutura inicial

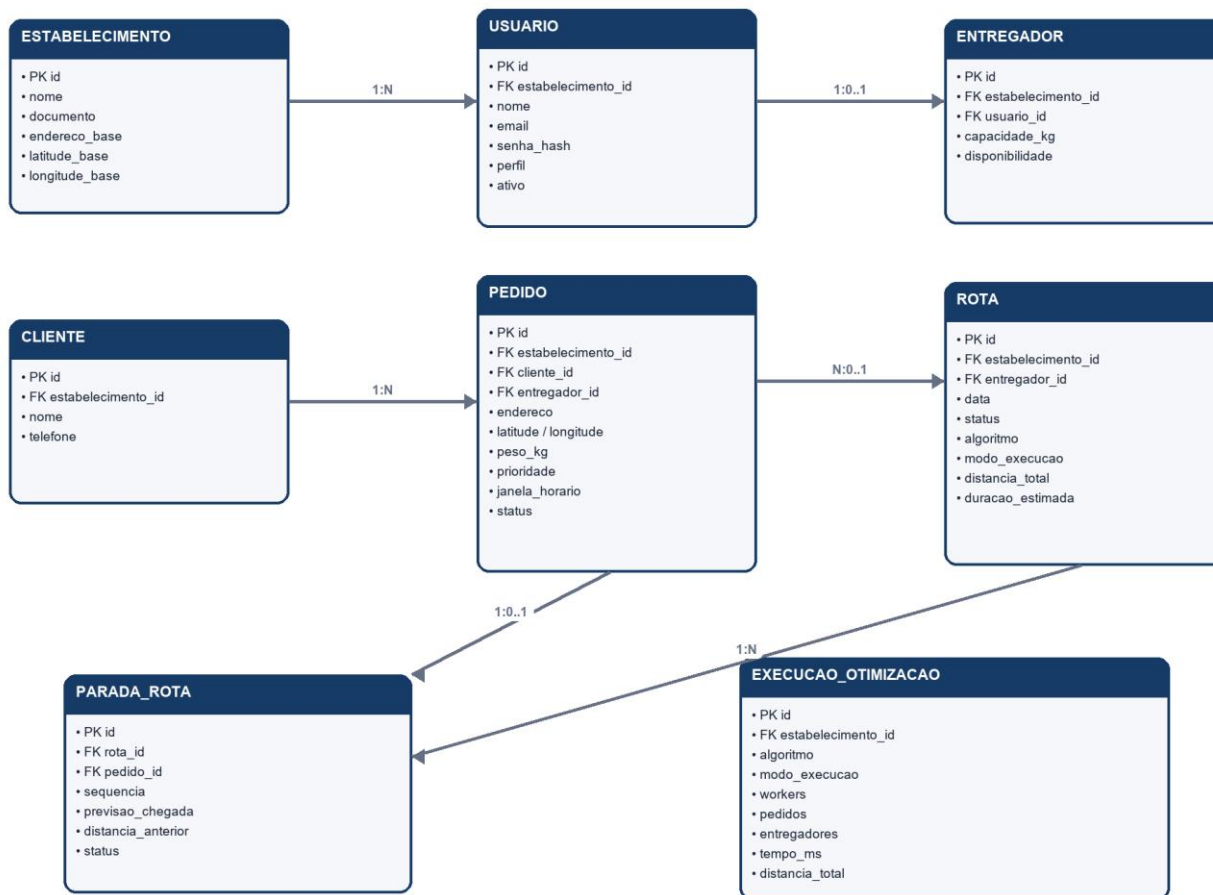


Figura 3 Modelo conceitual com chaves e cardinalidades

O modelo mantém o estabelecimento como limite organizacional. Um usuário pode possuir um cadastro de entregador. Cada pedido pertence a um cliente e pode ser atribuído a um entregador. Uma rota contém paradas ordenadas; cada parada referencia um pedido. As execuções de otimização registram quantidade de pedidos, entregadores, workers, tempo e distância total.

9 Modelo relacional

Tabela	PK	FK	Finalidade
establishments	id	-	Negócio, endereço e coordenadas do depósito
users	id	establishment_id	Credenciais, perfil e situação do usuário
couriers	id	establishment_id, user_id	Capacidade e disponibilidade do entregador
customers	id	establishment_id	Cliente associado ao estabelecimento
orders	id	establishment_id, customer_id, assigned_courier_id	Destino, peso, prioridade, janela e status
routes	id	establishment_id, courier_id	Rota diária, algoritmo, modo e totais
route_stops	id	route_id, order_id	Sequência e acompanhamento de cada parada
optimization_runs	id	establishment_id	Métricas de execuções sequenciais e paralelas

As restrições CHECK limitam perfis, estados e modos de execução. As chaves estrangeiras preservam a integridade entre usuários, pedidos, entregadores e rotas. Índices cobrem as consultas mais frequentes por estabelecimento, status, entregador, data e modo de execução.

10 Protótipos das telas principais

Os wireframes priorizam navegação direta e leitura em dispositivos móveis. A primeira sequência cobre autenticação, painel diário e cadastro de pedido. A segunda cobre geração de rotas, consulta da rota pelo entregador e atualização de status.

Protótipos principais
Fluxo administrativo em baixa fidelidade

RotaCerta

Acesso

Entrar no sistema

E-mail

Senha

Entrar

Perfis: administrador, atendente e entregador

RotaCerta

Painel do dia

Pedidos 38	Em rota 16	Entregues 19
-----------------------------	-----------------------------	-------------------------------

Operação

#1038 Ana Souza Pendente

#1037 João Lima Em rota

#1036 Carla Melo Entregue

Novo pedido **Gerar rotas**

RotaCerta

Novo pedido

Cliente

Endereço

Peso em kg **Prioridade**

2,50 Normal

Horário desejado

Cancelar **Salvar pedido**

Figura 4 Acesso, painel administrativo e cadastro de pedido

Protótipos principais

Geração de rotas e operação móvel do entregador

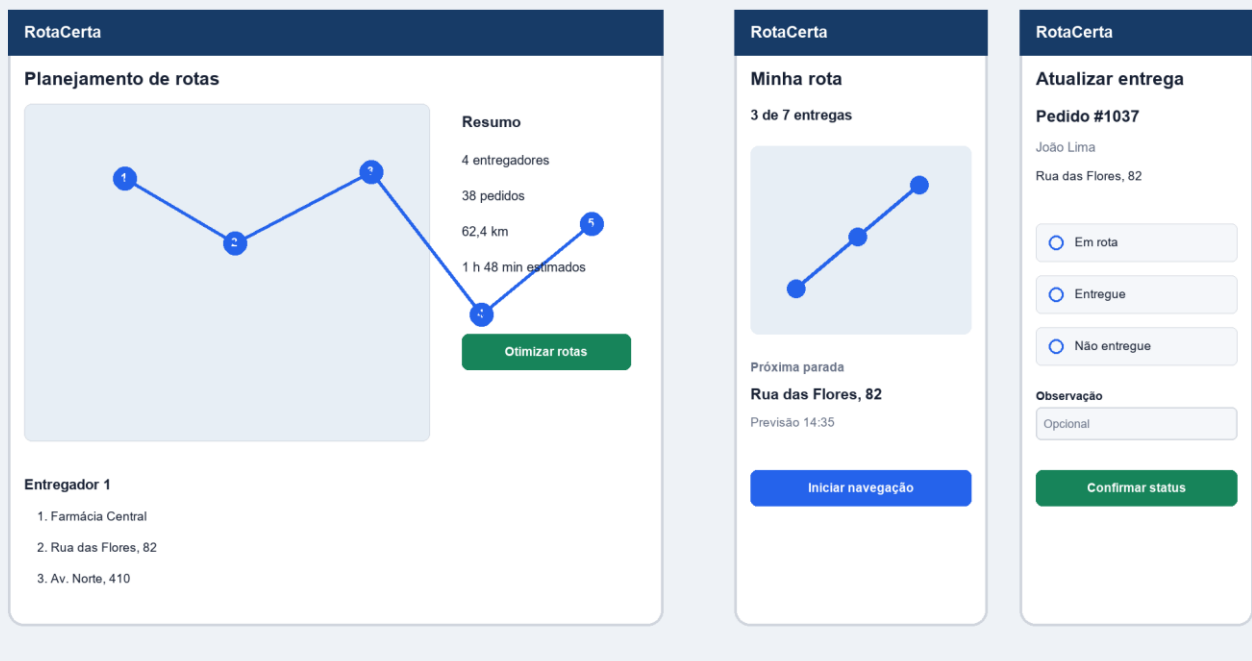


Figura 5 Planejamento de rotas e fluxo móvel do entregador

Tela	Requisito	Objetivo
Acesso	UC07	Login e identificação do perfil
Painel do dia	RF10	Pedidos, situação operacional e atalhos
Novo pedido	UC01 / RF02	Cliente, destino, carga, prioridade e janela
Planejamento	UC03 / RF04	Mapa, totais e comando de otimização
Minha rota	UC04 / RF06	Sequência, mapa e próxima parada
Atualizar entrega	UC05 / RF07	Transição de status e observação

11 Banco de dados e projeto no GitHub

Banco de dados criado

A estrutura inicial está implementada no arquivo `database/schema.sql` para PostgreSQL 16. O serviço db do arquivo `compose.yaml` executa o esquema automaticamente na primeira inicialização do volume. O modelo inclui oito tabelas, restrições de integridade e cinco índices iniciais.

1. Copiar `.env.example` para `.env`.
2. Executar `docker compose up --build`.
3. Aguardar o health check do PostgreSQL.
4. Consultar a API em `http://localhost:8000/health` e a documentação em `http://localhost:8000/docs`.

Estrutura do repositório

Caminho	Conteúdo
<code>backend/</code>	API FastAPI e limite do módulo de otimização
<code>backend/app/optimizer/</code>	Vizinho mais próximo, 2-opt e execução sequencial ou paralela
<code>backend/tests/</code>	Testes automatizados do núcleo de roteirização
<code>frontend/</code>	Aplicação React e TypeScript
<code>database/</code>	DDL, instruções e inicialização do PostgreSQL
<code>docs/sprint-02/assets/</code>	Diagramas e protótipos incorporados ao documento
<code>tools/documentation/</code>	Geração reproduzível dos materiais técnicos
<code>compose.yaml</code>	Orquestração local dos três serviços
<code>CONTRIBUTING.md</code>	Padrões de branch, commit e revisão

Repositório oficial. <https://github.com/alvccpj/rota-certa>

Branch	Finalidade
<code>master</code>	Versão estável e entregas aprovadas
<code>sprint/02-arquitetura-modelagem</code>	Integração da Sprint 02
<code>dev/alvaro</code>	Ambiente pessoal de Álvaro
<code>dev/arthur</code>	Ambiente pessoal de Arthur
<code>dev/vinicius</code>	Ambiente pessoal de Vinicius

O fluxo recomendado usa branches curtas por tarefa, pull request e revisão de outro integrante. As branches pessoais identificam ambientes acadêmicos, mas não substituem branches de feature, correção ou documentação.

12 Avaliação sequencial e paralela

A orientação da Sprint 01 exige manter a comparação entre as versões sequencial e paralela. A primeira implementação já executa vizinho mais próximo e refinamento 2-opt nos dois modos. As versões usam os mesmos pedidos, entregadores e coordenadas; a única variável controlada é o modo de execução.

Métrica	Cálculo ou unidade	Critério
Tempo de execução	Milissegundos	Média, mediana e percentil 95 de dez repetições
Distância total	Quilômetros	Confirma que a paralelização não altera a qualidade da rota
Speedup	T_{seq} / T_{par}	Ganho obtido com a execução paralela
Eficiência	$Speedup / workers$	Uso relativo dos processos disponíveis
Escala	Pedidos e entregadores	Cenários de 50, 100, 250 e 500 pedidos com 2, 4 e 8 entregadores

Cada cenário terá uma execução de aquecimento e dez repetições válidas. O relatório registrará processador, memória, sistema operacional, versão do código, número de workers e semente dos dados. A tabela `optimization_runs` armazenará cada medição para permitir auditoria e reprodução.

Evidência prática

O arquivo `backend/app/optimizer/routing.py` contém a heurística determinística, o cálculo de distância geodésica, o refinamento 2-opt e a execução com processos independentes. O endpoint `POST /optimizer/compare` retorna rotas, distâncias, tempos, workers, speedup e a confirmação de equivalência entre os modos. Cinco testes automatizados validam rota vazia, preservação das paradas, melhoria do 2-opt, resultado da otimização e equivalência sequencial/paralela.

13 Aderência às orientações da disciplina

Orientação	Situação	Evidência
Projeto integrado	Atendido	Um único sistema reúne interface, API, banco e otimização
Componente avançado	Atendido	Otimização e paralelização fazem parte da geração de rotas
Tecnologia prioritária	Atendido	Python no núcleo; JavaScript e TypeScript apenas na interface
Integração não superficial	Atendido	O endpoint compara os modos sobre os mesmos dados
Evolução prática	Atendido	Código executável e cinco testes automatizados no repositório
GitHub atualizado	Atendido	Branch da Sprint 02 e Pull Request número 1
Participação da equipe	Contínuo	Cada integrante deve registrar contribuições próprias nas próximas tarefas

Os requisitos de software completo, autenticação, perfis, funcionalidades integradas e demonstração final pertencem à evolução do semestre. A Sprint 02 estabelece a base verificável para essas implementações sem declarar como concluído o produto final.

Situação da Sprint 02

Entrega	Situação	Evidência
Arquitetura do sistema	Concluído	Tecnologias, responsabilidades, comunicação e componente avançado definidos
Diagrama de classes	Concluído	Classes, atributos, métodos e relacionamentos apresentados
MER	Concluído	Entidades, chaves e cardinalidades apresentadas
Modelo relacional	Concluído	Tabelas, PKs, FKs e finalidade documentadas
Protótipos	Concluído	Seis telas principais em baixa fidelidade
Banco de dados	Concluído	DDL PostgreSQL e inicialização via Compose
Motor de otimização	Concluído	Primeira versão sequencial/paralela com cinco testes
GitHub	Concluído	Estrutura, padrões, branch da Sprint e Pull Request número 1
Número do grupo	Pendente	Substituir [INFORMAR] na capa e no nome do arquivo antes do envio

Próxima etapa

A Sprint 03 deve validar o ambiente em uma máquina da equipe, confirmar a criação do PostgreSQL e integrar o frontend aos endpoints de saúde e comparação do otimizador. A equipe também deve transformar os casos de uso prioritários em tarefas semanais com critérios de aceite e registrar contribuições individuais no GitHub.